

JPA Entity Lifecycle

Chapter 04 - JPA and Hibernate

Sub-Chapter 04.01

What is JPA?	Spec that maps Java objects to DB tables
Lifecycle States	Transient, Persistent, Detached, Removed
EntityManager Ops	persist, find, merge, remove, contains
@Entity Basics	@Entity, @Id, @GeneratedValue, @Column
Persistence Context	First-level cache / identity map
Transactions and Flush	Transaction boundaries and flush modes

1 What is JPA?

Java Persistence API - a specification, not an implementation

JPA (Java Persistence API) is a standard **specification** (JSR 338) that defines how Java objects (POJOs) are mapped to relational database tables. It is **not** a library you run directly. Instead, JPA is implemented by providers such as Hibernate, EclipseLink, and OpenJPA. Think of JPA as the interface and Hibernate as the class that implements it.

Key idea: you annotate a plain Java class with JPA annotations, and the provider generates the SQL for you. You work with objects; the provider handles INSERT, SELECT, UPDATE, DELETE.

Java Example

```
// A minimal JPA entity
import jakarta.persistence.*;

@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name")
    private String name;

    // getters and setters ...
}
```

Key Rules

- JPA is a specification (javax/jakarta.persistence.*), NOT a framework itself.
- Hibernate is the most popular JPA provider; Spring Boot auto-configures it.
- SQL is generated by the provider, reducing boilerplate significantly.
- JPA entities must have a no-arg constructor (can be package-private or public).
- The @Table annotation is optional; defaults to the class name as table name.

Q1. What is the difference between JPA and Hibernate?

JPA is a specification (a set of interfaces/rules). Hibernate is a concrete implementation of that spec. You code to JPA interfaces; Hibernate does the real work.

Q2. Do I need to write SQL when using JPA?

For basic CRUD you write no SQL at all. For complex queries you can use JPQL (Java Persistence Query Language) or native SQL when needed.

Q3. Which dependency do I add for JPA in a Maven project?

Add spring-boot-starter-data-jpa (Spring Boot) or hibernate-core + jakarta.persistence-api manually.

Q4. Can one JPA entity map to multiple tables?

Yes, using @SecondaryTable or inheritance strategies, but it is advanced. Beginners should map one entity to one table.

Q5. Is JPA only for relational databases?

JPA is designed for RDBMS (MySQL, PostgreSQL, etc.). For NoSQL, different APIs exist (Spring Data MongoDB, etc.).

2 Entity Lifecycle States

Transient -> Persistent -> Detached -> Removed

Every JPA entity passes through four possible states during its lifetime. Understanding these states is fundamental to avoiding bugs like unexpected INSERTs, stale data, and LazyInitializationException.

State	Description
Transient	Object created with 'new', not yet known to any EntityManager. No DB row.
Persistent	Managed by the active EntityManager. Changes are tracked and synced to DB.
Detached	Was managed, but the EntityManager closed or detach() was called. Changes NOT tracked.
Removed	Marked for deletion. A DELETE will fire on flush/commit.

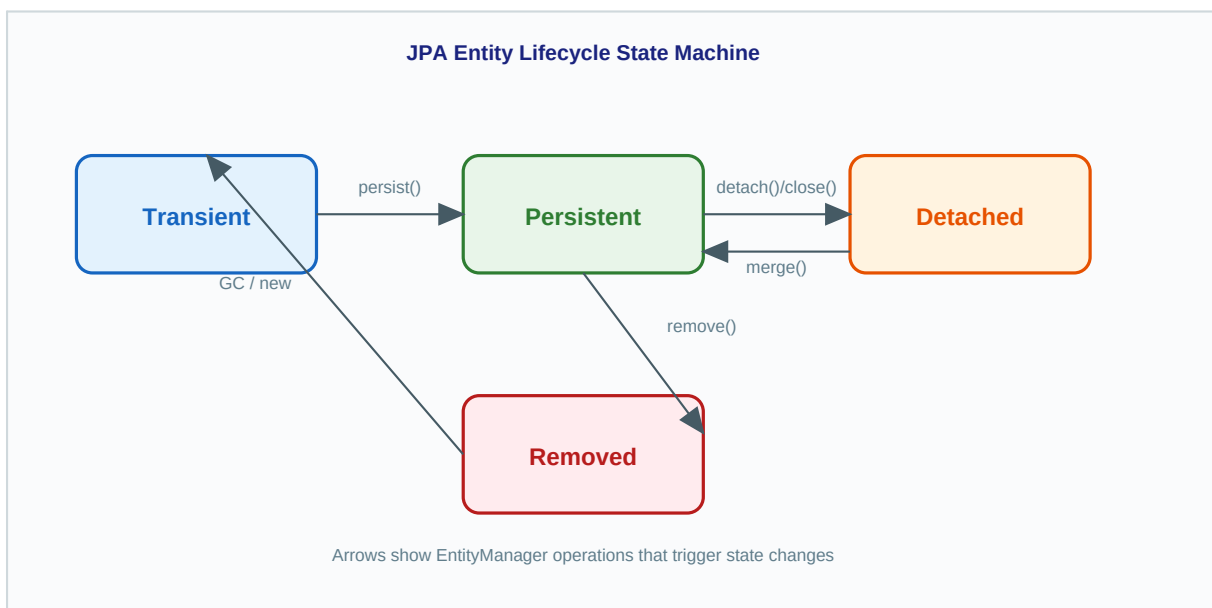


Figure 2.1 - JPA Entity State Machine (transitions via EntityManager operations)

Java Example - Observing State Transitions

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("myPU");
EntityManager em = emf.createEntityManager();

Student s = new Student();    // TRANSIENT
s.setName("Alice");

em.getTransaction().begin();
em.persist(s);                // PERSISTENT - now tracked
em.getTransaction().commit();

em.detach(s);                 // DETACHED - changes no longer tracked
s.setName("Alice Smith");     // change ignored by JPA

em.close();
```

Key Rules

- A Transient object has no identity in the DB and no association with any EntityManager.
- Persistent entities are inside the 'Persistence Context'. Changes auto-sync on flush.

- Detached entities can be reattached via `merge()`. The merge returns a NEW managed copy.
- Removed entities are still in memory until garbage collected after commit.
- Closing the `EntityManager` detaches ALL entities currently in that context.

Q1. What happens if you modify a Detached entity?

The change is ignored by JPA. To save it, call `em.merge(entity)` which returns a new Persistent copy.

Q2. Can a Transient entity become Removed?

No. Only Persistent entities can be passed to `remove()`. Attempting to remove a Transient entity throws an exception.

Q3. What makes an entity Persistent automatically?

If a Persistent entity has a `@OneToMany(cascade=PERSIST)` relationship and you add a Transient child to its collection, the child becomes Persistent automatically.

Q4. Is a Detached entity still in the database?

Yes. It still has a DB row. Detached only means the `EntityManager` is no longer watching it.

Q5. What state is an entity in after `commit()` without `close()`?

Still Persistent. The `EntityManager` stays open; the entity remains managed across transactions unless explicitly detached.

3 EntityManager Operations

persist, find, merge, remove, contains

EntityManager is the central API for interacting with the Persistence Context. It is the JPA equivalent of Hibernate's Session. You use it to save, load, update, and delete entities.

Method	What It Does	State Transition
<code>persist(entity)</code>	Makes a Transient entity Persistent. Schedules INSERT.	Transient -> Persistent
<code>find(Class, id)</code>	Loads entity by primary key. Returns null if not found.	DB row -> Persistent
<code>merge(entity)</code>	Copies state of Detached entity into a new Persistent copy.	Detached -> Persistent (new copy)
<code>remove(entity)</code>	Marks a Persistent entity for deletion. Schedules DELETE.	Persistent -> Removed
<code>contains(entity)</code>	Returns true if entity is currently managed (Persistent).	No state change
<code>detach(entity)</code>	Removes entity from context. Changes no longer tracked.	Persistent -> Detached
<code>flush()</code>	Writes all pending changes to DB (within transaction).	No state change
<code>refresh(entity)</code>	Reloads entity state from DB, discarding in-memory changes.	Persistent (refreshed)

Java Example - All Main Operations

```
em.getTransaction().begin();

// persist: save new entity
Student s = new Student("Bob");
em.persist(s);                // INSERT queued

// find: load by id
Student found = em.find(Student.class, 1L);

// merge: reattach detached entity
Student merged = em.merge(detachedStudent);

// remove: delete
em.remove(found);             // DELETE queued

em.getTransaction().commit();  // SQL fires here
```

Key Rules

- `persist()` requires a Transient entity; passing a Detached entity may throw `EntityExistsException`.
- `merge()` does NOT modify the passed object; it returns a NEW Persistent instance.
- `remove()` must be called on a Persistent entity. Detach it first if needed.
- `find()` checks the first-level cache before hitting the database.

- contains() is useful for guards: only call remove() if contains() returns true.

Q1. What is the return value of merge()?

merge() returns the managed Persistent copy. The original detached object passed in is NOT managed after the call.

Q2. What is the difference between find() and getReference()?

find() loads the entity immediately. getReference() returns a proxy and loads data only when a field is accessed (lazy load).

Q3. When does the INSERT SQL actually execute after persist()?

On flush() or when the transaction commits, whichever comes first.

Q4. Can I call remove() on a Detached entity?

No. You must first merge() it to get a Persistent copy, then call remove() on that copy.

Q5. What does flush() do exactly?

It synchronises the in-memory Persistence Context to the database within the current transaction. The transaction is NOT committed.

4 @Entity Annotation Basics

@Entity, @Id, @GeneratedValue, @Column

JPA uses annotations to describe how a Java class and its fields map to a database table and its columns. These four annotations are the absolute minimum you need to get started.

Annotation	Purpose	Required?
@Entity	Marks the class as a JPA entity (mapped to a table)	Yes
@Id	Marks the field as the primary key	Yes
@GeneratedValue	Delegates PK generation to the DB (AUTO_INCREMENT, SEQUENCE, etc.)	No, Not Recommended
@Column	Customises column name, nullable, length, unique constraints	No (optional)
@Table	Specifies custom table name or schema	No

Java Example - Full Annotated Entity

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", nullable = false, length = 50)
    private String firstName;

    @Column(unique = true)
    private String email;

    protected Employee() {} // required no-arg constructor

    public Employee(String firstName, String email) {
        this.firstName = firstName;
        this.email = email;
    }
}
```

Key Rules

- @Entity marks the class; @Id marks the primary key field - both are mandatory.
- GenerationType.IDENTITY uses DB auto-increment; SEQUENCE uses a DB sequence object.
- @Column is optional; without it JPA maps the field to a column with the same name.
- The no-arg constructor must exist (any visibility). JPA uses it to instantiate entities.
- Primitive types (int, long) cannot be null; use wrapper types (Integer, Long) for nullable PKs.

Q1. Do I need @Column if the column name matches the field name?

No. @Column is optional. JPA uses the field name as the column name by default.

Q2. What is the difference between GenerationType.IDENTITY and AUTO?

IDENTITY relies on the database auto-increment feature. AUTO lets the provider choose the best strategy for the underlying DB.

Q3. Can I use a composite primary key with JPA?

Yes, using `@EmbeddedId` with an `@Embeddable` class, or `@IdClass`. Both are more advanced topics.

Q4. What happens if the no-arg constructor is missing?

JPA will throw an exception at startup or when trying to instantiate the entity from a DB row.

Q5. Can I annotate fields or getter methods?

Both work. If you annotate fields, JPA accesses fields directly. If you annotate getters, JPA uses property access. Be consistent; do not mix both in the same entity.

5 Persistence Context

First-level cache and identity map

The Persistence Context is the in-memory container managed by the EntityManager. It acts as a **first-level cache** (also called the identity map). When you load an entity with find(), JPA first checks this cache before hitting the DB. If the entity is already there, it returns the same Java object - no extra SELECT.

The Persistence Context guarantees that within a single EntityManager scope, loading the same entity by the same primary key always returns the **exact same object reference**. This is called the identity guarantee.

Java Example - First-Level Cache in Action

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

Student a = em.find(Student.class, 1L); // hits DB
Student b = em.find(Student.class, 1L); // returns cached object, NO SQL

System.out.println(a == b);           // true (same reference!)

em.getTransaction().commit();
em.close();
// cache is gone after close()
```

How the Identity Map Works

Concept	Explanation
First-Level Cache	Automatic per-EntityManager cache. Cannot be disabled. Cleared on close().
Identity Map	Maps (EntityClass, primaryKey) -> Java object. Guarantees one object per PK.
Second-Level Cache	Optional, shared across EntityManagers. Requires explicit configuration.
Dirty Checking	JPA compares current entity state to a snapshot taken at load time to detect changes.

Key Rules

- The first-level cache is scoped to a single EntityManager. Different EMs have separate caches.
- Within the same EM, loading the same PK twice always returns the identical Java object (==).
- The cache is automatically cleared when the EntityManager is closed.
- Dirty checking compares the current field values against a snapshot captured at load/persist time.
- To force a DB re-read, call em.refresh(entity) which overwrites the cached state.

Q1. Is the first-level cache always active?

Yes. It is built into every EntityManager and cannot be disabled. It is transparent to the developer.

Q2. What is the difference between first-level and second-level cache?

First-level is per-EntityManager (per request/transaction). Second-level is shared across all EntityManagers in the factory, requires configuration, and is optional.

Q3. Can the first-level cache cause stale data problems?

Yes, if another process updates the DB directly. Call em.refresh(entity) to reload from DB, or use optimistic locking with @Version.

Q4. Does the cache persist across transactions?

Within the same EntityManager, yes. Each new EntityManager starts with an empty cache.

Q5. What is dirty checking and when does it run?

Dirty checking compares entity fields to their original snapshot. It runs at flush time (before commit or on explicit flush()) and generates UPDATE SQL only for changed fields.

6 Transaction Boundaries and Flush

When SQL actually executes

JPA operations like `persist()` and `remove()` do **not** immediately execute SQL. They schedule work inside the Persistence Context. The actual SQL is sent to the DB when the context is **flushed**. Flushing happens automatically before commit, or you can trigger it manually with `em.flush()`.

Understanding this is critical for debugging. If you expect a row in the DB after `persist()` but before `commit()`, you will not see it (in a different session) until flush/commit.

Java Example - Explicit Transaction Control

```
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();

try {
    tx.begin();

    Student s = new Student("Carol");
    em.persist(s);          // no SQL yet

    em.flush();             // INSERT fires NOW (still inside transaction)

    s.setName("Carol M");// dirty checked at next flush

    tx.commit();            // final flush + COMMIT sent to DB
} catch (Exception e) {
    tx.rollback();          // undo everything
} finally {
    em.close();
}
```

Flush Modes

Flush Mode / Call	Behaviour
FlushModeType.AUTO	Default. JPA flushes before executing a query if needed, and before commit.
FlushModeType.COMMIT	JPA only flushes when the transaction commits. Useful for read-heavy work.
<code>em.flush()</code>	Manual flush within a transaction. SQL fires but transaction is NOT committed.

Key Rules

- Always wrap writes in a transaction. JPA will throw `TransactionRequiredException` otherwise.
- `tx.commit()` triggers a flush automatically before the COMMIT is sent to the DB.
- `tx.rollback()` cancels all pending changes. The Persistence Context should be discarded after rollback.
- `em.flush()` is useful when you need the DB to see the changes before the transaction ends (e.g., to run a native query that depends on the new rows).
- In Spring, `@Transactional` handles begin/commit/rollback automatically; you rarely call them manually.

Q1. Does `persist()` immediately insert a row in the DB?

No. It schedules an INSERT. The SQL fires on flush (which happens at commit by default).

Q2. What happens if I never commit a transaction?

If the transaction is rolled back or the connection closes, all scheduled changes are discarded. No data reaches the DB.

Q3. Can I flush multiple times within one transaction?

Yes. Each flush sends accumulated SQL to the DB but does not commit. You can flush, do more work, and flush again, then commit once.

Q4. What is the risk of FlushModeType.COMMIT?

Queries within the transaction may return stale results because unflushed changes are not yet visible to the DB query engine.

Q5. In Spring Boot, do I need to manage transactions manually?

Rarely. Annotate your service method with `@Transactional` and Spring handles begin, commit, and rollback automatically.